

A Project Report on SocialNEURON: AI Social Media Content Automation Platform

**In fulfillment of the requirement for the 4th Semester of
Master of Computer Application Programme**

Course Code: MCA-401

Course Name: Software Development Project

Academic Year / Session: [Enter Academic Year / Session]

Submitted by

[Name of the Learner]

Enrollment No.: [Enter Enrollment Number]

Under the Guidance of

[Name of the Project Guide]

Learner Support Centre: [Name of Learner Support Centre]

Location: [Location]

Dr. Babasaheb Ambedkar Open University

Certificate of Originality from the Guide

This is to certify that the project report entitled **SocialNEURON: AI Social Media Content Automation Platform** submitted to Dr. Babasaheb Ambedkar Open University in partial fulfillment of the requirement for the award of the degree of Master of Computer Applications is an original work carried out by [Name of the Learner], Enrollment No. [Enrollment Number], under the supervision of [Name of the Project Guide].

The matter embodied in this project is a genuine work done by the learner and has not been submitted either to this University or to any other University or Institute for fulfillment of the requirement of any course of study.

Signature of the
Learner: of the Signature of the Guide:

Name:	Name:
Address:	Designation:
Enrollment	Address:
Date:	No.: Date:

Acknowledgement

I express my sincere gratitude to my project guide, faculty members, and the Learner Support Centre for their valuable guidance and support during the development of this software project. Their suggestions helped in shaping the project scope, improving the system design, and preparing this report in a structured manner.

I am thankful to Dr. Babasaheb Ambedkar Open University for providing the MCA-401 Software Development Project framework, which encouraged practical application of software engineering, database design, web development, and project management concepts.

I also thank my family, peers, and all users who provided feedback during the development and testing of SocialNEURON. Their inputs helped improve the usability, workflow, and overall quality of the project.

Declaration

I hereby declare that this project report titled **SocialNEURON: AI Social Media Content Automation Platform** is an original software development project prepared by me for MCA-401. The work presented in this report is based on my own analysis, design, implementation, testing, and documentation of the project. All references used have been acknowledged in the References section.

Signature of Learner: _____

Name: [Name of Learner]

Enrollment No.: [Enrollment Number]

Date: _____

Table of Contents

Chapter / Section	Title	Page No.
Certificate	Certificate of Originality from the Guide	
Acknowledgement	Acknowledgement	
Declaration	Declaration	
Synopsis	Project Proposal / Synopsis	
Chapter 1	Introduction	
Chapter 2	Requirement Analysis	
Chapter 3	Logical Design	
Chapter 4	Physical Design	
Chapter 5	Implementation	
Chapter 6	Testing	
References	References	
Appendix	Diagram Prompts, Sample Screens, and Supporting Material	

Project Proposal / Synopsis

Title of the Project

SocialNEURON: AI Social Media Content Automation Platform.

Introduction and Objectives of the Project

SocialNEURON is a web based software application designed to help businesses, creators, and marketing teams plan, generate, review, and verify social media content. The platform allows users to define brand details during onboarding, upload or manually create content topics, generate AI assisted drafts, review content, manage email alerts, and maintain a publication history.

The primary objective of the project is to create a practical, scalable, and user-friendly web application that solves the real-world problem of consistent social media content production. The project demonstrates concepts of web development, database management, user authentication, workflow automation, AI integration, file import, email notification, and report-oriented project documentation.

Project Category

The project belongs to the Application Oriented and Database Oriented categories. It is a web based application developed using Python and Django with a relational database backend.

Tools, Platform, Hardware and Software Requirements

Category	Specification
Programming Language	Python
Web Framework	Django 5.2
Frontend	HTML, Tailwind CSS, HTMX, JavaScript
Database	SQLite for development, compatible with relational databases for production
AI Integration	Ollama local API for content generation
Task	Celery and Redis

Processing

Data Import Pandas and OpenPyXL for
Excel / CSV schedule upload

Email Django email backend

Operating Linux / Windows compatible
System development environment

Hardware Minimum 8 GB RAM, modern
CPU, 2 GB free disk space;
higher RAM recommended for
local AI models

Industry / Client

This project is developed as an academic software development project. The target users are small businesses, founders, content teams, freelancers, agencies, and creators who need a structured system for content planning and review.

Methodology

The project follows an iterative software development approach. Requirements were identified from the workflow of content teams. The system was divided into modules such as authentication, onboarding, topic generation, schedule upload, AI draft generation, review workflow, email integration, contact inquiry, and dashboard reporting. Each module was developed, tested, and improved incrementally based on observed user flows.

Expected Output

The expected output is a working web application where a user can register, define brand profile, generate or upload content topics, create AI generated drafts, approve or reject content, mark published content with verification link, receive email alerts, and manage content history.

Conclusion

SocialNEURON demonstrates a real-world application of software development concepts. It combines database design, web interface design, authentication, file processing, AI integration, task automation, and testing practices to build a functional content automation platform.

Chapter 1: Introduction

1.1 Brief Idea About the Project

SocialNEURON is an AI powered social media content automation system. It helps users move from idea generation to content review and publication tracking in a single web application. A typical user first creates an account, completes onboarding by entering brand information, then uses the dashboard to generate topics, upload Excel schedules, review generated content, and manage published history.

The system is not limited to a single platform. It supports content planning for platforms such as LinkedIn, Instagram, X, Facebook, TikTok, and multiple content types such as post, reel, carousel, story, thread, and short. This flexibility makes the application more suitable for modern social media workflows.

1.2 Objective of the Project

The objectives of SocialNEURON are:

- To provide a secure user registration and login system.
- To capture brand profile information such as niche, audience, platforms, tone, and goals.
- To generate topic ideas using AI according to selected platform and content type.
- To allow manual topic creation and Excel based bulk schedule upload.
- To generate AI assisted content drafts using brand voice and topic context.
- To provide review actions such as approve, reject, regenerate, comment, and mark as published.
- To send email notifications to configured recipients.
- To maintain dashboard analytics and history of published content.
- To collect contact inquiries from the public home page.

1.3 Scope of the Project

The scope includes complete user and content workflow management for a social media automation platform. It covers authentication, onboarding, dashboard reporting, schedule import, topic generation, draft generation, review management, published link verification, email alert routing, contact inquiry submission, and basic administrative management through Django admin.

The system can be extended in future to include direct publishing APIs, advanced analytics, team roles, payment plans, file storage, and deeper AI prompt management.

1.4 Existing System

Many small businesses manage social media content using spreadsheets, messaging apps,

manual notes, and disconnected AI tools. In such workflows, topic planning, AI content generation, approval, and publication tracking are spread across multiple tools. This causes duplication, inconsistent brand tone, missed deadlines, and difficulty in tracking approved or published posts.

1.5 Proposed System

The proposed system centralizes the content workflow into one web application. A user can create topics manually, generate AI topics, upload schedules through Excel, run the AI workflow for due drafts, approve or reject generated content, and manually verify published posts. Brand profile information influences the generated content so the output remains consistent with the business identity.

1.6 Platform Used: Hardware and Software

Type	Details
Backend Platform	Django web framework using Python
Frontend Platform	Django templates, Tailwind CSS, HTMX, JavaScript
Database	SQLite during development
Background Jobs	Celery with Redis
AI Provider	Ollama local text generation API
Development Tools	Python virtual environment, Git, Django management commands

1.7 Project Location

The project is developed locally as a Django web application. The source code is organized under the project folder named `SoicalNEURON`. The application can be deployed to a web server or cloud platform after configuring database, email backend, Redis, and environment variables.

Chapter 2: Requirement Analysis

2.1 Introduction

Requirement analysis identifies what the system must do and what constraints it must satisfy. For SocialNEURON, requirements were derived from the content operations workflow of users who need to create regular social media posts with consistent tone and controlled approvals.

2.2 Tools Used for Requirement Gathering

The following tools and methods were used for requirement gathering:

- Observation of social media planning workflows.
- Review of spreadsheet based content calendars.
- Study of AI-assisted content generation use cases.
- Module analysis based on Django app structure and database models.
- Feedback driven iteration during implementation.

Functional Requirements

Requirement ID	Description	Module
FR-01	User can sign up, login, logout, reset password, and change password.	Authentication
FR-02	User can enter and update brand profile information.	Onboarding / Settings
FR-03	User can generate topic ideas using query, tone, audience, platform, content type, and count.	Topic Generator
FR-04	User can add a manual topic	Manual topic

	with date, time, platform, and content type.	
FR-05	User can upload content schedule through Excel or CSV.	Upload
FR-06	System can generate AI content drafts for due posts.	AI Workflow
FR-07	User can approve, reject, regenerate, comment on, and verify posts.	Review
FR-08	System can send email notifications to configured recipients.	Email Integration
FR-09	User can view dashboard statistics and published history.	Dashboard / History
FR-10	Visitor can submit a contact inquiry from the public home page.	Contact

Non-Functional Requirements

Requirement	Description
Usability	The interface must be easy to understand with clear cards, forms, and workflow actions.

Security	Password hashing, session-based access control, CSRF protection, and scoped user data are required.
Reliability	Cron should avoid regenerating already generated content unnecessarily.
Maintainability	Code should be divided into Django apps, services, models, views, and templates.
Scalability	The design should allow migration from SQLite to production relational databases.
Extensibility	The system should support future platform APIs, richer analytics, and subscription plans.

2.3 Problems in Existing System

Existing manual systems lack workflow continuity. Topic ideas may be generated in one tool, reviewed in another, scheduled in a spreadsheet, and published manually without reliable history. Email alerts are often not centralized, and brand voice is usually not stored in a structured manner. This leads to operational delays, inconsistent content quality, and weak tracking.

2.4 Conclusion

The requirement analysis shows that an integrated web application is needed to support planning, AI generation, review, and publication verification. SocialNEURON satisfies these requirements by combining content operations and AI generation in a single workflow.

Chapter 3: Logical Design

3.1 Introduction

Logical design describes the system in terms of processes, data flow, entities, and user interactions. It defines how the system should behave before focusing on physical implementation details.

3.2 Data Flow Diagram

0th Level DFD

The 0th level DFD represents SocialNEURON as a single system interacting with users, email recipients, AI service, and file uploads.

Graph Prompt: Generate a 0th level DFD for "SocialNEURON AI Social Media Content Automation Platform". External entities: User, Email Recipient, Ollama AI Service, Excel/CSV File. Main process: SocialNEURON System. Data flows: login data, brand profile, content topics, uploaded schedule, AI prompt/response, review decision, email notification, published link.

Figure 3.1: 0th Level Data Flow Diagram

1st Level DFD

Graph Prompt: Generate a 1st level DFD for SocialNEURON with processes: Authentication, Onboarding and Brand Profile, Topic Generation, Schedule Upload, AI Draft Generation, Review Workflow, Email Notification, Contact Inquiry, Dashboard and History. Show data stores: User, BrandProfile, PostSchedule, ContentItem, EmailRecipient, ContactInquiry, PostComment.

Figure 3.2: 1st Level Data Flow Diagram

2nd Level DFD: Review Workflow

Graph Prompt: Generate a 2nd level DFD for the Review Workflow module. Include processes: view pending drafts, approve post, reject post, regenerate content, add review note, copy generated content, mark as published with link. Include data stores: PostSchedule and PostComment. Include user as external entity.

Figure 3.3: 2nd Level Data Flow Diagram for Review Workflow

3.3 ER Diagram

The main entities of the system are User, BrandProfile, PasswordResetToken, PostSchedule, ContentItem, PostComment, EmailRecipient, ContactInquiry, BrandVoice, and PlatformIntegration.

Graph Prompt: Generate an ER diagram for a Django based SocialNEURON system. Entities and attributes: User(id, email, password, is_active, is_staff, created_at, updated_at),

BrandProfile(id, user_id, brand_name, niche, audience, platforms, tone, goals), PasswordResetToken(id, user_id, token_hash, expires_at, used_at), PostSchedule(id, user_id, date, topic, category, platform, post_type, priority, published_link, tone, generated_content, status, engagement_score, readability_score, virality_score, sentiment, is_duplicate), ContentItem(id, user_id, topic, platform, post_type, generated_content, status), PostComment(id, post_id, user_id, content), EmailRecipient(id, email, name, is_active), ContactInquiry(id, name, email, company, subject, message, status), BrandVoice(id, name, document_content), PlatformIntegration(id, platform, webhook_url, setup_email, is_active). Show one-to-one User to BrandProfile, one-to-many User to PostSchedule, one-to-many PostSchedule to PostComment, one-to-many User to ContentItem.

Figure 3.4: Entity Relationship Diagram

3.4 Use Case Diagram

Graph Prompt: Generate a UML use case diagram for SocialNEURON. Actors: Visitor, Registered User, Admin, AI Service, Email Service. Use cases: register, login, reset password, submit contact inquiry, complete onboarding, update brand profile, generate topics, add manual topic, upload schedule, run cron, review content, approve content, reject content, regenerate content, add notes, mark published, manage email recipients, view dashboard, view history, manage admin records.

Figure 3.5: Use Case Diagram

3.5 Activity Diagram

Graph Prompt: Generate an activity diagram for the SocialNEURON content lifecycle. Start with user login, check onboarding status, create or upload topic, schedule topic, run cron, generate AI content, save pending draft, send email alert, user reviews content, decision approved/rejected/regenerate, if approved then user manually posts content and enters published link, system marks as published, end with history updated.

Figure 3.6: Activity Diagram

3.6 Conclusion

The logical design shows that SocialNEURON follows a modular workflow. Data flows from user input to schedule records, then to AI generated drafts, review decisions, email notifications, and final publication history.

Chapter 4: Physical Design

4.1 Introduction

Physical design describes how the logical design is implemented using database tables, Django models, templates, services, views, and background task functions.

4.2 Database Design

Table / Model	Purpose	Important Fields
User	Stores authentication identity.	email, password, is_active, is_staff
BrandProfile	Stores onboarding and brand voice information.	brand_name, niche, audience, platforms, tone, goals
PasswordResetToken	Stores password reset tokens securely.	user, token_hash, expires_at, used_at
PostSchedule	Stores scheduled topics and generated content status.	date, topic, platform, post_type, generated_content, status, scores
ContentItem	Stores generated draft items for approval workflow.	topic, platform, post_type, generated_content, status
PostComment	Stores review post,	user,

	notes on content, scheduled created_at posts.
EmailRecipient	Stores configured email, email name, recipients for is_active alerts.
ContactInquiry	name, Stores home email, page contact company, form subject, submissions. message, status
BrandVoice	Stores uploaded or name, entered brand document_ voice content reference text.
PlatformIntegration	Stores platform, optional webhook_u integration rl, configuration. setup_email, is_active

Normalization

The database design separates users, brand profiles, scheduled posts, generated content, comments, and email recipients into separate models. This reduces duplication and supports maintainability. User-related records are linked through foreign keys or one-to-one relations. Contact inquiries and email recipients are independent operational records.

4.3 Module Design

Module	Files / Components	Description
Authentication	auth/ auth_service.py, users/models.py,	User creation, login, password reset,

	login/reset templates	password change.
Onboarding	users/ onboarding.py, onboarding template, settings page	Brand profile creation and update.
Topic Generator	core/views.py, content_generati on.py, topic_generator. html	Generates topic ideas based on query, tone, audience, platform, and type.
Schedule Upload	schedule_import .py, upload.html	Imports Excel or CSV schedule into PostSchedule.
AI Draft Workflow	core/tasks.py, content_generati on.py	Generates content and scoring for due scheduled posts.
Review Center	review_dashboar d.html, post_detail.html	Approve, reject, regenerate, comment, and publish verification.
Email Integration	email_integratio n.html, approval_service .py	Manages recipients and sends review alerts.
Dashboard	dashboard.html, dashboard_settin gs.html	Displays metrics and settings.
Contact	home.html, ContactInquiry model	Stores public inquiry submissions.

4.4 Input / Output Design

Input	Source	Output
Email and password	Signup / Login forms	User account and session
Brand profile	Onboarding / Settings	Brand context voice
Topic query, platform, post type	Topic Generator	AI topic suggestions
Excel / CSV file	Upload page	Scheduled draft topics
Review action	Review Center	Status change to approved, rejected, or published
Email recipient list	Email Integration page	Notification routing list
Contact details	Home page	Contact inquiry record

4.5 Conclusion

The physical design maps the logical workflow to Django models, templates, services, and views. It supports modular growth and separates business logic from presentation wherever practical.

Chapter 5: Implementation

5.1 Introduction

The implementation is carried out using Django. The project contains a main Django project package, application modules, templates, migrations, services, and supporting scripts. The implementation emphasizes readable workflows, secure user handling, and practical integration with AI and email services.

5.2 Process Description

User Authentication Process

Users can sign up with email and password. Passwords are hashed using Django password hashers. Users can login, logout, request a reset link, reset password through token, and change password from the dedicated settings page.

Onboarding and Brand Profile Process

After signup, users enter brand name, niche, target audience, preferred platforms, writing tone, and content goals. This data is used to build a reusable brand voice payload for AI prompts.

Topic Generation Process

The topic generator accepts query, tone, audience, platform, content type, and number of topics. The server calls the content generation service, which prepares a prompt for Ollama and parses returned lines into topic objects. The results are displayed with score, virality, tone, audience, platform, and content type information.

Schedule Upload Process

Users can upload Excel or CSV files containing columns such as date, topic, tone, audience, platform, priority, and post_type. The import service reads the file and creates scheduled draft records.

AI Draft Generation Process

The cron workflow checks today and tomorrow's draft records. If a draft does not already contain generated content, the system generates content using AI, assigns scores, changes status to pending, saves a content item, and sends email notifications to active recipients.

Review and Publishing Process

Users review pending drafts, approve or reject them, regenerate content, add comments, and mark approved content as published by entering a live URL. Published posts appear in the history page.

5.3 Output and Reports

The application produces the following outputs:

- Dashboard statistics for total, draft, pending, approved, rejected, and published content.
- AI generated topic ideas.
- AI generated social media content drafts.
- Email notifications for review-ready content.
- Published history with platform, tone, score, and link information.
- Contact inquiry records for public visitors.

Screenshot and Graph Placeholders

Screenshot Prompt: Capture the dashboard page showing summary cards, upcoming schedule, recent activity, and workspace settings card. Label as Figure 5.1: Dashboard View.

Screenshot Prompt: Capture the topic generator page showing query, tone, audience, platform, content type, count slider, and generated results. Label as Figure 5.2: Topic Generator View.

Screenshot Prompt: Capture the review dashboard showing pending, approved, and rejected columns. Label as Figure 5.3: Review Center View.

Screenshot Prompt: Capture the post detail page showing generated content, score rings, review actions, and notes. Label as Figure 5.4: Post Detail View.

Screenshot Prompt: Capture the email integration page showing recipients and actions. Label as Figure 5.5: Email Integration View.

Screenshot Prompt: Capture the public home page contact form. Label as Figure 5.6: Contact Inquiry Form.

5.4 Conclusion

The implementation successfully creates a functional end-to-end content automation workflow. It integrates user management, AI generation, review controls, file import, email notifications, and public contact handling into a single Django application.

Chapter 6: Testing

6.1 Introduction

Testing verifies that the system works according to the requirements. The project was tested through module level testing, integration testing, user interface testing, form validation testing, and workflow testing.

6.2 Types of Testing Performed

Unit Testing

Individual functions such as password validation, schedule parsing, topic generation parsing, and contact submission handling can be tested independently.

Integration Testing

Integration testing checks whether modules work together. Examples include signup to onboarding flow, topic generation to draft saving, schedule upload to AI generation, and review approval to published history.

System Testing

The complete system is tested as a web application by navigating through all major workflows from login to content publication.

Validation Testing

Form validation checks required fields in contact inquiry, login, signup, reset password, topic creation, schedule upload, and published link entry.

Regression Testing

Regression testing verifies that updates such as contact form, settings page, email recipient deletion, topic platform selection, and score ring fixes do not break older workflows.

Test Cases

Test Case ID	Scenario	Expected Result	Status
TC-01	User signs up with valid email and password.	Account created and redirected to Pass onboarding.	

TC-02	User enters mismatched signup passwords.	Error message is displayed.	Pass
TC-03	User updates brand profile from settings page.	Brand profile is updated.	Pass
TC-04	User generates topics for Instagram Reel.	Results show selected platform and content type.	Pass
TC-05	User uploads Excel schedule.	Rows are added as draft scheduled topics.	Pass
TC-06	User runs cron for due draft.	AI content is generated and status becomes pending.	Pass
TC-07	User runs cron again for already generated content.	Content is not regenerated unnecessarily.	Pass
TC-08	User adds note on post detail.	Comment is saved and displayed.	Pass
TC-09	User adds email recipient.	Recipient appears in email integration list.	Pass
TC-10	User deletes email recipient.	Recipient is removed.	Pass
TC-11	Visitor submits contact inquiry.	Inquiry is saved and success message displayed.	Pass
TC-12	User marks	Error message is	Pass

post as
published displayed.
without URL.

6.3 Conclusion

Testing confirms that the core modules of the SocialNEURON system meet the defined requirements. The application handles common workflows such as authentication, onboarding, topic generation, schedule upload, AI draft generation, review, email alerts, and contact inquiry submission.

References

1. Dr. Babasaheb Ambedkar Open University, MCA-401 Software Development Project Guidelines, <https://baou.edu.in/assets/pdf/MCA-401.pdf>
2. Django Software Foundation, Django Documentation, <https://docs.djangoproject.com/>
3. Celery Documentation, <https://docs.celeryq.dev/>
4. Pandas Documentation, <https://pandas.pydata.org/docs/>
5. Tailwind CSS Documentation, <https://tailwindcss.com/docs>
6. HTMX Documentation, <https://htmx.org/docs/>
7. Ollama Documentation, <https://ollama.com/>

Appendix A: Report Formatting Notes

This report draft is prepared according to the following guide-based formatting expectations:

- A4 page size.
- Left margin 3.0 cm, right margin 2.0 cm, top margin 2.55 cm, bottom margin 2.55 cm.
- Normal body text: Times New Roman, 12 pt, justified.
- Paragraph heading: Times New Roman, 14 pt, left aligned.
- Chapter heading: Times New Roman, 20 pt.
- Code font: Courier New, 10 pt where applicable.
- All figures and tables should be numbered and titled.
- Page numbers should be inserted at bottom center in the final word processor document.

Appendix B: Additional Diagram Prompts

Architecture Diagram Prompt: Draw a layered architecture diagram for SocialNEURON showing Browser UI, Django Views, Services, Models, SQLite Database, Celery Worker, Redis Broker, Ollama AI API, and Email Backend. Show request and response arrows.

Database Schema Prompt: Create a normalized relational database schema diagram for the SocialNEURON models with primary keys, foreign keys, and cardinality symbols.

Deployment Diagram Prompt: Create a deployment diagram showing user browser, Django web server, database server, Redis server, Celery worker, email server, and Ollama service.

Sequence Diagram Prompt: Create a sequence diagram for "Generate AI Draft" with participants User, Browser, Django View, PostSchedule, ContentGenerationService, Ollama, ContentItem, EmailService, EmailRecipient.

Appendix C: Sample Environment Variables

```
DJANGO_SECRET_KEY=change-me
DJANGO_DEBUG=True
DJANGO_ALLOWED_HOSTS=localhost,127.0.0.1
EMAIL_HOST=smtp.example.com
EMAIL_PORT=587
EMAIL_USE_TLS=True
EMAIL_HOST_USER=user@example.com
EMAIL_HOST_PASSWORD=password
DEFAULT_FROM_EMAIL=demo@socialneuron.local
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=llama3
```